Firmware Upgrade Campaign Assignment

Oct 24, 2026

Overview

The position you are interviewing for involves building a fully integrated, Gen AI powered and eventually autonomous device management system that manages a device's life cycle for configuration, provisioning, firmware upgrades, container life cycle orchestration and remote diagnostics. It is the single point of entry to gateway commands for management. The system will manage tens of millions of gateway modems across the AT&T Fiber and Fixed Wireless blueprint.

As part of this assignment, you will build a small device management controller that runs a firmware upgrade campaign across a fleet of simulated home gateways. No prior knowledge of telecom device management protocols is required. The device interactions have been translated to REST and JSON.

Time budget: we expect about 8–10 hours of focused effort. You have 5 calendar days to submit. We recommend not to spend significantly more; we value judgement about scope.

AI tools: You are welcome to use AI coding assistants. The design and the submission must be your own work, though: no help from other people. In the follow-up system design or coding interview, expect to walk us through your code and extend it live while sharing your screen with focus on design decision and coding implementation choices made.

Hosting: It must be runnable on the AWS or Azure free tier, or locally via Docker Compose. Include instructions.

The campaign

To communicate with a home gateway the controller sends it a *connection request* (a wake-up nudge). The device then calls back into the controller, and only during that session can the controller issue commands. A firmware upgrade is asynchronous: the controller sends a *Download* command with a unique `commandKey`; the device acknowledges, downloads the image, reboots, and sometime later opens a new session and reports *TransferComplete* carrying the same `commandKey` and a result code. The controller must correlate the two across the reboot.

Interfaces

Device → Controller (you implement the Controller side API to process the incoming request)

- `POST /devices/{deviceId}/inform`: The device opens a session. Body: `{ deviceId, firmwareVersion, event: "BOOT" | "CONNECTION_REQUEST" | "PERIODIC" | "TRANSFER_COMPLETE", transferComplete?: { commandKey, faultCode, startTime, completeTime } }`. The response may contain zero or more pending commands for the device, e.g. `{ commands: [ { type: "DOWNLOAD", commandKey, url, targetVersion } ] }`.
- `POST /devices/{deviceId}/command-response`: The device acknowledges a command. Body: `{ commandKey, status: "ACCEPTED" | "REJECTED", reason? }`.

Controller → Device (provided by the simulator, so you make the call to these APIs)

- `POST {simulatorUrl}/devices/{deviceId}/connection-request`: Returns `202` if the nudge was delivered. Delivery does **not** guarantee the device will call back.
- `GET {simulatorUrl}/devices?version={firmwareVersion}`: Lists the fleet as `[ { deviceId, firmwareVersion } ]`. Use it to find the devices a campaign targets. NOTE: DO NOT use this API to poll for the version without handling a BOOT and/or TransferComplete to correlate success or failure for the upgrade campaign.

Operator API (you implement)

- `POST /campaigns`: Body: `{ targetVersion, imageUrl, deviceFilter: { currentVersion }, maxConcurrent, failureThresholdPercent }`.
- `GET /campaigns/{id}`: Returns progress: counts per device state, failure rate and campaign status.
- `POST /campaigns/{id}/pause` and `/resume`

The simulator

We provide am AI generated Python simulator example, **not guaranteed to work but as hint or advice** for how devices behave, `device_simulator.py`, for 1,000–10,000 devices. We also use it to evaluate the behavior as a 100% success is NOT expected in real production, devices may fail at various stages of the upgrade. Feel free to use as is with working changes or write equivalent apps using AI in another language of your choice. Run it locally or in a container with `CONTROLLER_URL` pointing at your controller and `DEVICE_COUNT` set to the fleet size; every device starts on firmware `1.0.0`. All devices send a periodic inform every few minutes. The fleet is fixed by a seed, so your results are repeatable. As mentioned above, it is configured to misbehave realistically:

- Some devices are offline and never respond to connection requests.
- Some devices reject the Download command.
- Some devices accept the command and then report a TransferComplete with a non-zero fault code.
- Some devices reboot and come back on the new firmware but **never send TransferComplete**.
- Some devices come back on the **old** firmware with a BOOT event and no TransferComplete.
- Some callbacks are duplicated or arrive late.
- Reboot time varies from a few seconds to several minutes.

Requirements

1. Create and run a campaign that upgrades every eligible device to the target version.
2. Track each device through a well-defined state machine (for example: PENDING → NOTIFIED → DOWNLOAD_SENT → ACCEPTED → REBOOTING → SUCCEEDED / FAILED / TIMED_OUT). Define and document your own states and transitions.
3. Correlate each TransferComplete with the original Download using `commandKey`. Duplicates must be handled idempotently.
4. Decide and document how you treat devices that come back without a TransferComplete. Hint: the reported `firmwareVersion` is evidence.

5. Enforce `maxConcurrent` in-flight upgrades.
6. Automatically halt the campaign if the failure rate exceeds `failureThresholdPercent` after a minimum sample size.
7. Apply timeouts at each waiting state, with sensible retry behaviour. Note that some steps are safe to retry and some are not.
8. Controller state must survive a controller restart mid-campaign.
9. Expose metrics for number of request sent, errored, success, etc. Report by status code, endpoint type.

Helpful Hints

We are not worried about time, it can take 5 mins or even more to run. Don't concern yourself with running on a larger instance. In AWS free tier, their micro or even nano instances are enough.

Pay more attention to the core, the REST APIs, mini scheduler, response and status reporting, the state machine etc. Not to the peripherals.

Do not forget observability. Expose metrics.

And finally, keep it simple. Layering can complicate or stretch time boundaries.

Deliverables

- Source repository, with a README covering how to run it, the architecture, the state machine diagram and the key design decisions and trade-offs.
- Document config.yaml or equivalent with tuning parameters decided upon.
- A short run report from a campaign against the simulator: final counts per state and anything surprising.
- For a good bonus: A one-page note: **what would change to run this across 5, 10 or 15 million devices** with multiple controller instances? Would you use the "nudge" REST API or some other way? How would track, record and report on different status during and after the campaign ?

Evaluation criteria

- % Correctness (it should work!!!)
- % Completeness (meet the requirements)
- Observability quality
- Clean, readable code with comments (sanitize AI's verbose comments, we know this domain!!)
- Design choices with an eye on the future (is your design flexible)

Follow up

Please submit the deliverables described to the recruiter/hiring manager. If we choose to move forward, one of your interviews build on it depending on the skew of your submission.

Good luck!!!